

COMP2054-ADE

The “Master Theorem”

Lec12

Master Theorem (MT)

Consider recurrence relations of the form

$$T(n) = a T(n/b) + f(n)$$

- Designed for “divide and conquer” in which problems are divided into ‘a’ instances of a problem of size n/b .
- Aim is to be able to quickly express the “big-Oh family” behavior of $T(n)$ for various cases of the values of a and b , and the scaling behavior of $f(n)$.

It does not cover all cases, but does cover many useful cases.

Master Theorem

- No proof needed in this module.
 - Though you are expected to have some understanding of its structure, and why it is reasonable
- Need to learn it, and how to apply it!
- Suggest to generate and try many examples
- Note: it is also a very good example of the vital importance of understanding
 - The Big-Oh family
 - Logs with arbitrary bases – e.g. what is $\log_8(2)$?

Motivations

- Consider the special case that $f(n) = 0$
- $T(n) = a T(n/b)$ with $T(1) = 1$
 - $T(b) = a$
 - $T(b^2) = a^2$
 - $T(b^3) = a^3$
- So $T(b^k) = a^k$
- Exercise (offline): prove by induction

Motivations

- Consider the special case that $f(n) = 0$
 - $T(n) = a T(n/b)$ with $T(1) = 1$
 - Gives $T(b^k) = a^k$
- Put $n = b^k$ then
 - (for clarity in the superscripts write " $\log_b$ " as " $\log_b$ ")
 - $a^k = (b^{\log_b(a)})^k$
 - Note: used the identity that $a = b^{\log_b(a)}$
 - Then we have (swapping the order of the exponents)
 - $a^k = (b^k)^{\log_b(a)}$
 - $a^k = (n)^{\log_b(a)}$
- So we finally get $T(n) = n^{\log_b(a)}$

Motivations

- Consider the special case that $f(n) = 0$
 - $T(n) = a T(n/b)$ with $T(1) = 1$
 - Gives $T(n) = n^{\log_b(a)}$
- But now suppose $f(n) = n^c$ for some c
- We can ask which term dominates
 - the recurrence or the $f(n)$?
- So need to compare the values of c and $\log_b(a)$

Master Theorem (MT): Cases

$$T(n) = a T(n/b) + f(n)$$

Results are split into 3 cases, according to comparing the growth rate of $f(n)$ to $n^{\log_b(a)}$

- Case 1: $f(n)$ “grows slower”. Recurrence term dominates. “Solution ignores f ”
- Case 2: $f(n)$ grows same – up to log factors – “mix of recurrence with a, b , and also the $f(n)$ term”
- Case 3: $f(n)$ grows faster. “Solution ignores recurrence terms and a, b ”

MT: Case 1 : recurrence wins

$$T(n) = a T(n/b) + f(n)$$

$f(n)$ is $O(n^c)$ with $c < \log_b a$

Note: it is " $<$ " not " $\leq$ " and it is a "big-Oh"

Then $T(n)$ is $\Theta(n^{\log_b a})$

That is, $T(b^k)$ grows as $b^{k \log_b a} = a^k$,
as expected from earlier

MT: Case 1: Example

$$T(n) = 2 T(n/2) + d$$

$$a = 2, b=2 \quad \text{so } \log_b(a) = \log_2(2) = 1$$

$f(n)$ is $O(1)$ which is $O(n^c)$ with $c = 0$
and so we have that $c < \log_b(a)$

Hence MT gives that $T(n)$ is $\Theta(n)$

MT: Case 2 : a “draw”

$$T(n) = a T(n/b) + f(n)$$

if

$$f(n) \text{ is } \Theta(n^c (\log n)^k)$$

with $c = \log_b a$ and $k \geq 0$

(Note: it is “ $c =$ ” and Big-Theta)

Then $T(n)$ is $\Theta(n^c (\log n)^{k+1})$

Note the growth depends on both the recurrence, a, b , and also depends on f via k .

MT: Case 2: Example

$$T(n) = 2 T(n/2) + 3n + 5$$

$f(n)$ is $\Theta(n (\log n)^k)$

with

$$c = \log_2 2 = 1,$$

and

$$k=0$$

Then $T(n)$ is $\Theta(n \log n)$

(Same as merge sort of previous lecture)

MT: Case 3 : $f(n)$ wins

$$T(n) = a T(n/b) + f(n)$$

$f(n)$ is $\Omega(n^c)$ with $c > \log_b a$

Notice: it is " $c > ..$ " and big-Omega!

And $f(n)$ satisfies the "regularity condition"

$$a f(n/b) \leq k f(n) \quad \text{for some } k < 1$$

Then $T(n)$ is $\Theta(f(n))$.

Growth is dominated by $f(n)$ and so a, b of the recurrence are not used.

MT: Case 3 : Example

$$T(n) = 2 T(n/2) + n^2$$

$f(n)$ is $\Omega(n^c)$ with $c = 2 > \log_b a = \log_2 2 = 1$

Also, $f(n)$ satisfies the “regularity condition”

$$2 f(n/2) = 2 (n/2)^2 \leq k f(n) \quad \text{with } k=1/2$$

Then $T(n)$ is $\Theta(n^2)$

Regularity Condition (case 3)

- $a f(n/b) \leq k f(n)$ for some $k < 1$
- Suppose $f(n) = d n^c$ then we need
 - $a d (n/b)^c \leq k d n^c$ for some $k < 1$
 - $a / b^c \leq k$ for some $k < 1$
 - $a / b^c < 1$
 - $a < b^c$
 - Now take $\log_b$ of both sides
 - Need $\log_b(a) < c$
 - Which is already satisfied for case 3 to apply.
 - So is not a new condition in this case (needed for more complex cases)
- (Note: this is included for completeness; usually not relevant)

MT Example

- $T(n) = 4 T(n/2) + d n$ with $T(1)=1$
- $a=4, b=2$
- so $\log_b a = \log_2 4 = \log_2 2^2 = 2$
- Note that $f(n)$ is $O(n^c)$ with $c=1$
 - Hence $c < 2$, and is big-Oh
- Hence is case 1.
- Hence is $\Theta(n^2)$
 - Matches the exact solution in previous lecture.

Summary: Master Theorem

Master theorem: Given $T(n) = a T(n/b) + f(n)$ and $T(1)=1$.

Case 1: "Recurrence wins":

If $f(n)$ is $O(n^c)$ for some c , with $c < \log_b(a)$

then $T(n)$ is $\Theta(n^{\log_b(a)})$

Case 2: "Recurrence and f amplify each other"

If $f(n)$ is $\Theta(n^c (\log n)^k)$ for some $k \geq 0$, and with $c = \log_b(a)$

then $T(n)$ is $\Theta(n^c (\log n)^{k+1})$

Case 3: " f wins"

If $f(n)$ is $\Omega(n^c)$ for some c , with $c > \log_b(a)$

(and we need f to satisfy a "regularity condition")

then $T(n)$ is $\Theta(f(n))$

Expectations

- Know and understand the Master Theorem (MT)
 - Be able to apply it to examples
 - (Do not need to be able to prove the MT itself!)
- May well be asked to solve a recurrence relation exactly, and then also to solve it using the Master Theorem